

Software processes:

A structured set of activities required to develop a software system. These include:

- **Specification:**

Define what the software should do.

- **Design and implementation:**

Build the software according to specifications.

- **Validation:**

Ensure the software works as the customer expects.

- **Evolution:**

Update the software as customer needs change.

→ A software process model is an abstract representation of how these steps are carried out.

⇒ **Plan driven and agile processes:**

• Plan-Driven:

All steps are planned in advance and followed strictly.

• Agile:

More flexible; planning is done in small (methods) steps, making it easier to adjust based on changing requirements.

Plan-driven methods

- High criticality
- Junior developers
- Requirements do not change often.
- Large number of developers.
- Culture that demands order.

Agile methods

- Low criticality
- Senior developers
- Requirements change often.
- Small number of developers.
- Culture that responds to change.

=> Software process models:

• The waterfall model:

Plan driven model where steps are done in sequence.

• Incremental development:

Software is developed in smaller sections, allowing for fast delivery and feedback.

• Reuse oriented software engineering:

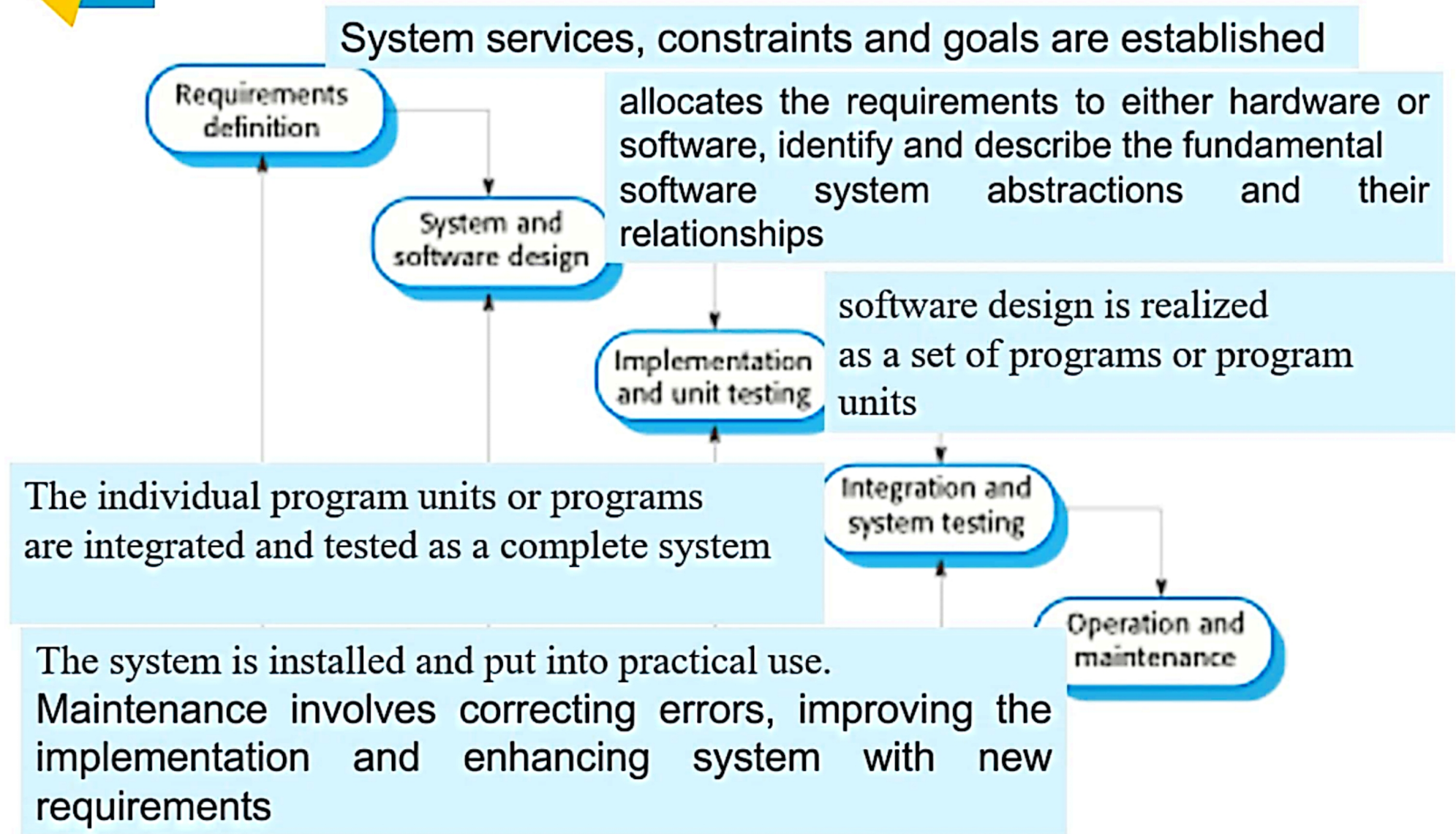
Use existing software components to build new systems.

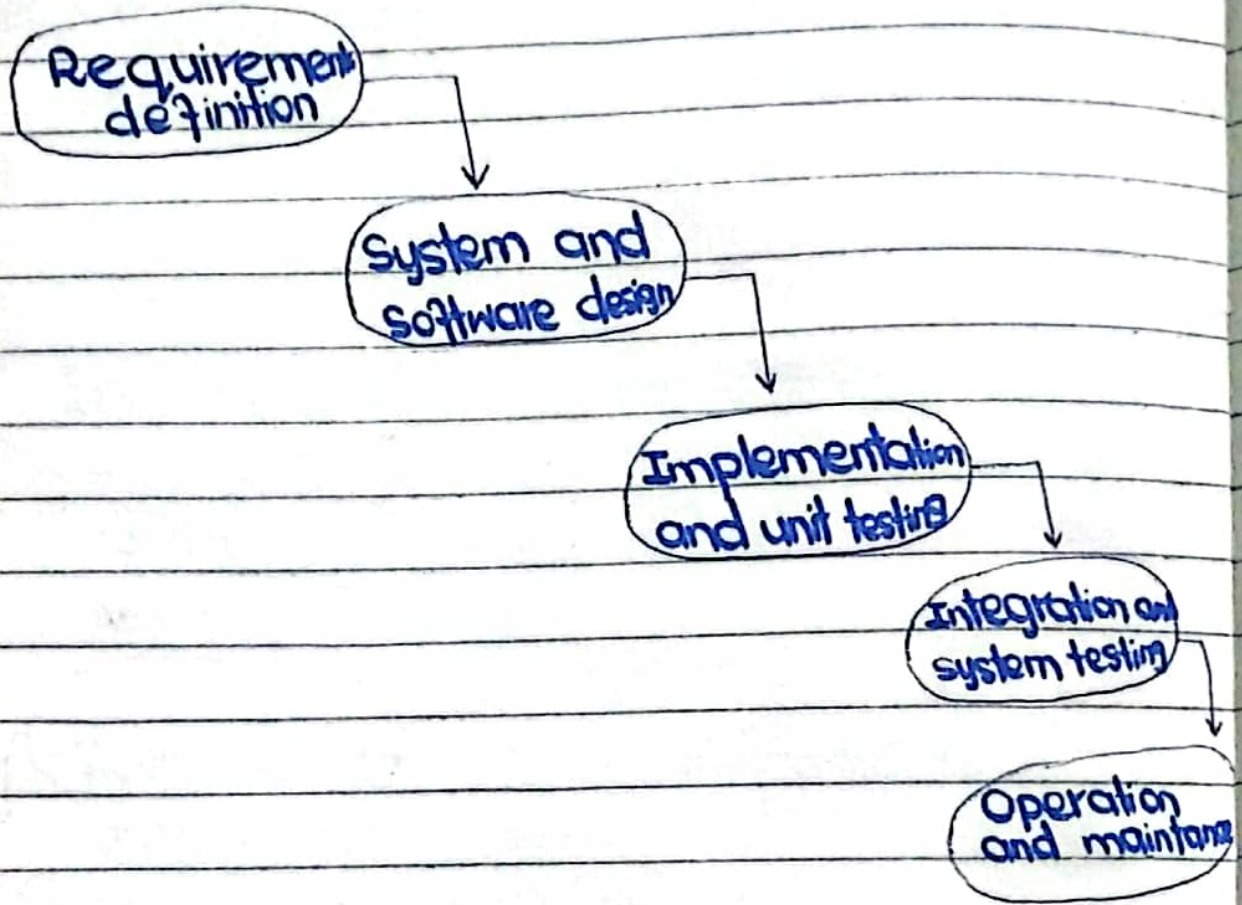
THE WATERFALL METHOD:

A sequential model where each phase must be completed before the next begins.



The waterfall model





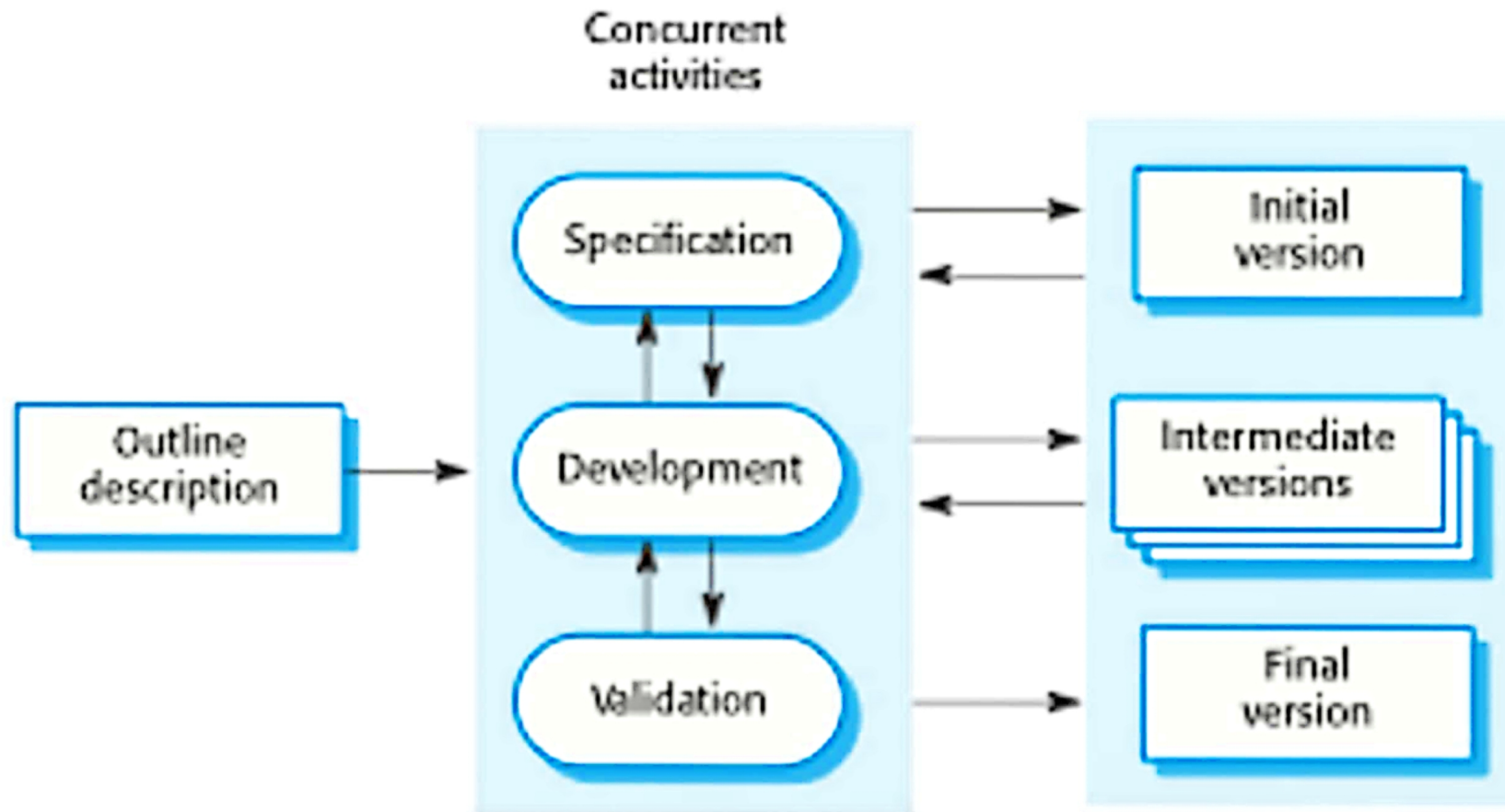
Waterfall model problems:

- Hard to accommodate changes once development starts.
- Suitable for projects with well-understood requirements.

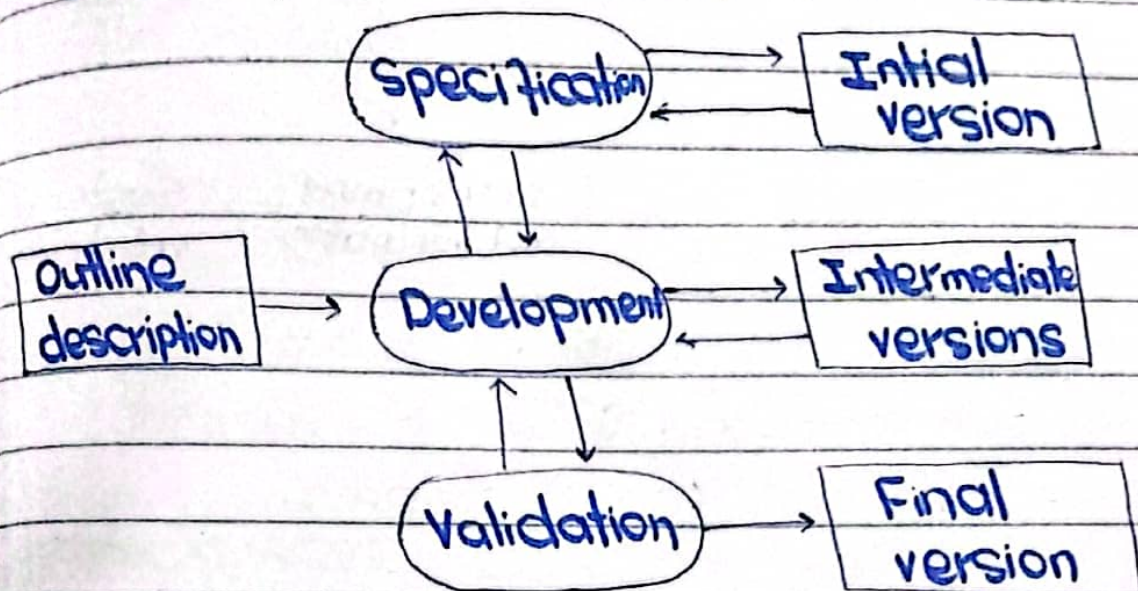
INCREMENTAL DEVELOPMENT



Incremental development



It involves creating an initial version and gradually improving it based on user feedback.



Incremental development benefits:

- Easier to make changes based on customer feedback.
- Allows faster delivery of parts of the software.

Incremental development problems:

The process might lack visibility, and system structure may degrade over time.

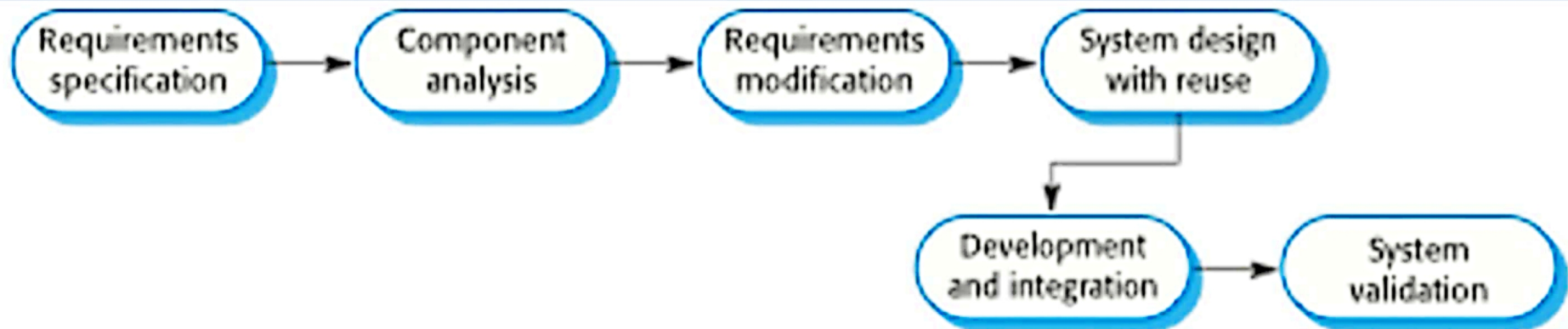
REUSE-ORIENTED SE:



Reuse-oriented software engineering

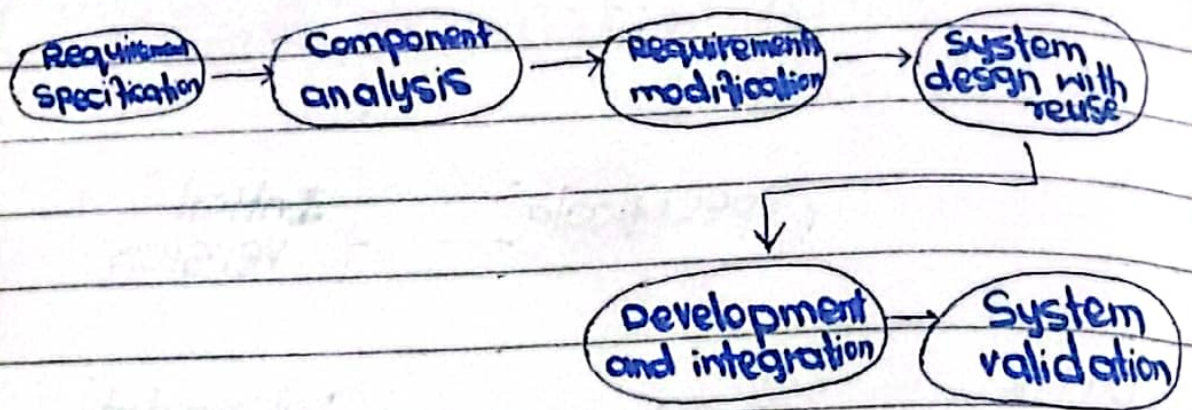
Component analysis: a search is made for components to implement that specification. Usually, there is no exact match

Requirements modification the requirements are analyzed using information about the components that have been discovered. They are then modified



System design with reuse: Framework of the system is designed, or an existing framework is reused by taking into account the components that are reused. If reusable components not available, some new software may have to be designed

Development and integration Software that cannot be externally procured is developed, and the components and COTS systems are integrated to create the new system.



Systems are built by integrating existing components or commercial off-the-shelf (COTS) systems.

Reuse oriented SE: Pros and cons:

Pros:

- Reduce cost and risk.
- Fast delivery.

Cons:

- May not meet all user needs perfectly.
- Limited control over component updates.

=> Software process activities:

• Specification:

Defining what the software will do.

• development:

writing the software.

• validation:

Ensuring the software works correctly.

• Evolution:

Modifying (Modifying) the Software to meet new needs.

=> Software design and implementation:

Involves turning the system specification into an actual working program.

• Design inputs:

→ Architectural design:

Define the system's main components.

→ Interface design:

Define how components will communicate.

→ Component design:

Detail how each part of the system will work.

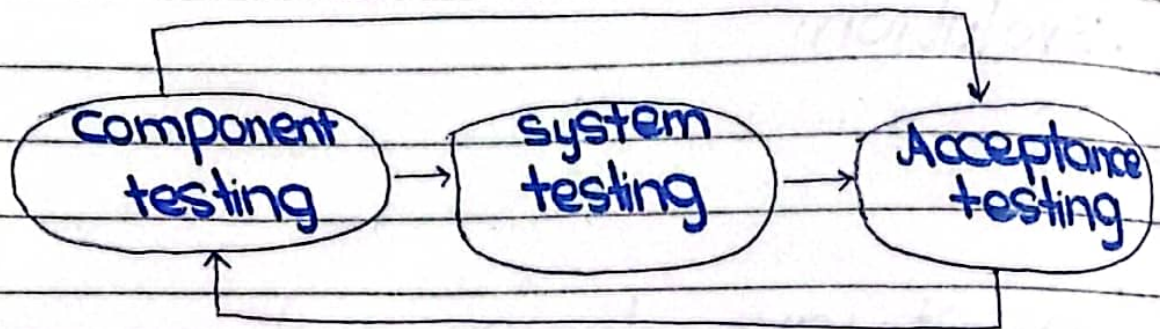
→ Database design:

Organize the system's data.

=> Software validation:

Ensures the software meets the requirements and works correctly.

=> Stages of testing:



=> Testing stages:

• Development or component testing:

Individual components are tested.

• System testing:

The whole system is tested.

• Acceptance testing / alpha testing:

The software is tested with real customer data.

• Beta testing:

When a system is to be marketed as a software product, a testing process called 'beta testing' is often used. Beta testing involves delivering

a system to several potential customers who agree to use that system. They report the problem to system developers.

⇒ Software evolution.

As business needs change, software needs to be updated. Evolution means ongoing updates, making the distinction between development and maintenance less important.

⇒ Coping with change.

• Change avoidance:

Anticipate change early e.g. by creating a prototype.

• Change tolerance.

Design the system to easily accommodate changes, typically using incremental development.

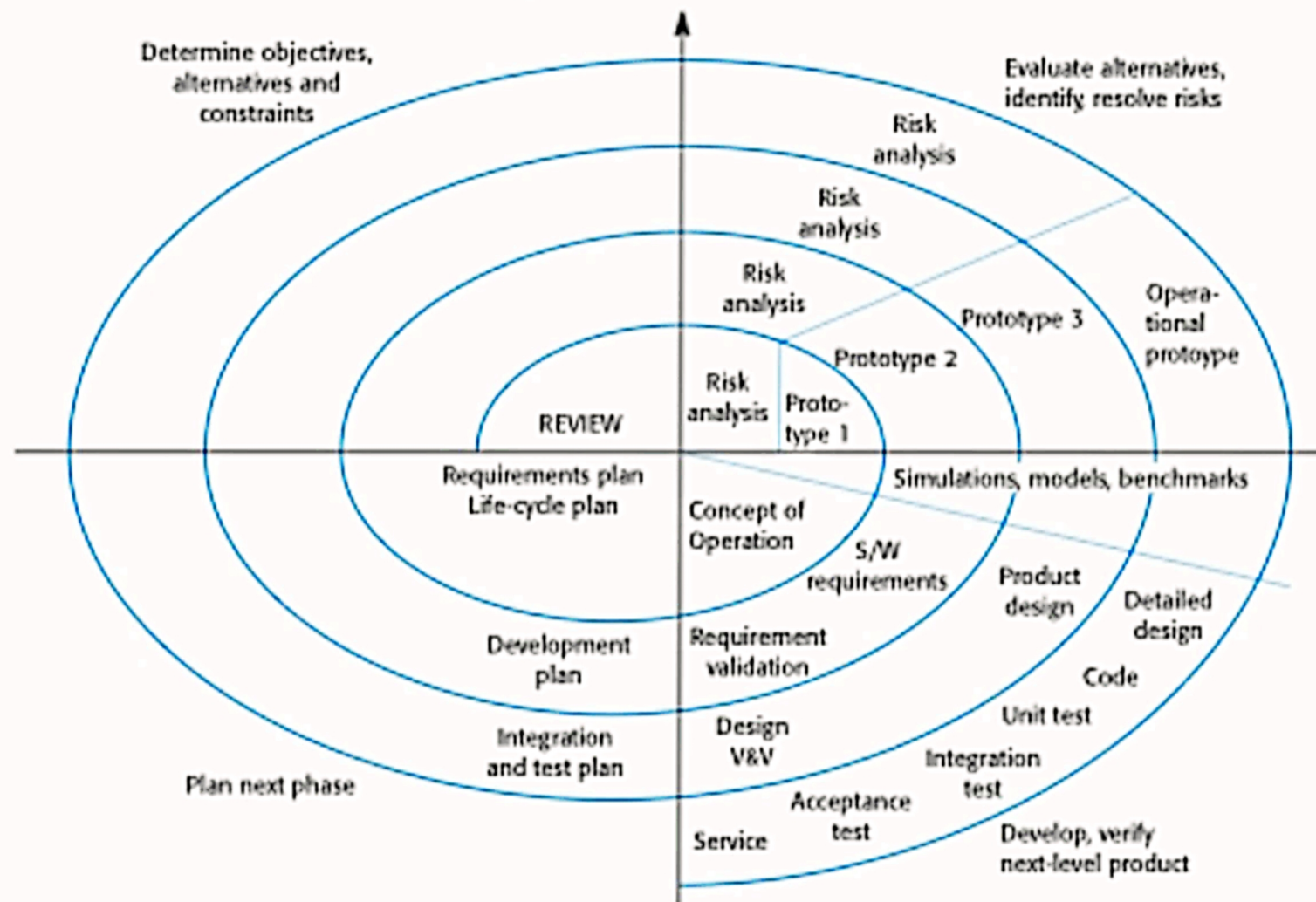
⇒ Bohem's spiral model: (Risk driven)

Focuses on assessing and reducing risks. Phases are flexible, adapting to the needs of each stage.

Phases are flexible, adapting to the needs of each stage.



Boehm's spiral model of the software process





Spiral model sectors

- ❑ Objective setting
 - ❑ Specific objectives for the phase are identified. Project risks are identified
- ❑ Risk assessment and reduction
 - ❑ Risks are assessed and activities put in place to reduce the key risks. For example, if there is a risk that the requirements are inappropriate, a prototype system may be developed.
- ❑ Development and validation
 - ❑ A development model for the system is chosen which can be any of the generic models.
 - ❑ For example, throwaway prototyping may be the best development approach if user interface risks are dominant. If the main identified risk is sub-system integration, the waterfall model may be the best development model to use.
- ❑ Planning
 - ❑ The project is reviewed, and the next phase of the spiral is planned.



Spiral model usage

- ❑ Spiral model has been very influential in helping people think about iteration in software processes and introducing the risk-driven approach to development.
- ❑ In practice, however, the model is rarely used as published for practical software development.

=> Rational Unified Process (RUP):

Combines elements from different models (waterfall, incremental, etc.)

=> RUP phases:

• Inception:

Define the business case.

• Elaboration:

Build understanding and plan the system.

• Construction:

Develop and test the system.

• Transition:

Deploy the system to the users.

=> RUP good practice:

• Develop software iteratively:

Manage requirements carefully.

Use visual models (e.g.; UML diagrams).

Control changes through proper management tools.

